Day 8~9 예습 — RAG (참고자료 → 임베딩 → 검색)

> 이 문서는 **읽기용이 아니라 준비용**이야. 내일 아침에 이거 한 번 읽고 시작하면, 개념 설명 없이 바로 손을 댈 수 있게 하려고 만들었어.
>
> **읽는 법:** 코드 블록에 `# ← 네가 칠 곳` 이라고 적힌 데는 **답이 없어.** 일부러 비워뒀어. 미리 채우려고 하지 마. 그냥 "아, 여기가 내 몫이구나" 정도만 보고 넘어가.

---

0. 지금 우리 프로그램은 어떻게 생겼나 (복습)

RAG를 붙이려면 **어디에 붙일지**를 알아야 해. 지금 흐름부터 다시 보자.

```
[강사가 API 호출]  POST /feedbacks/generate  {child_id: 1}
        │
        ▼
  ① DB에서 아이 정보 + 수업기록 꺼냄          (feedbacks_router.py)
        │
        ▼
  ② build_prompt(child, records)  →  긴 글 한 덩어리  (prompt_service.py)
        │
        ▼
  ③ generate_feedback(prompt)     →  Gemini가 JSON으로 답함 (llm_service.py)
        │
        ▼
  ④ json.loads(...)  →  Feedback(...)  →  DB 저장    (feedbacks_router.py)
```

RAG는 ②번에 끼어들어. 프롬프트를 조립할 때, 참고자료 조각을 하나 더 끼워 넣는 거야.

지금 `build_prompt`는 이렇게 생겼어 (실제 네 코드):

```
prompt = f"""[역할]
당신은 아동 줄넘기 수업 강사를 돕는 AI 피드백 코치입니다.

[아이 프로필]
- 이름: {child.name}
...
[최근 수업 기록]
{records_text}

[작성 규칙]
...
"""
```

Day 10에 여기에 **칸이 하나 더** 생길 거야:

```
[참고자료]
{rag_context}          # ← 검색해서 찾아온 조각이 여기 들어감
```

그게 RAG의 끝이야. 이 칸을 채우는 게 목적이고, Day 8~9는 전부 **이 칸에 넣을 글자를 구하는 과정**이야.

> 이 문장 하나만 기억해도 돼: **RAG =** `{rag_context}` **칸에 넣을 글을 찾아오는 일.**

---

1. 오늘 남은 것 — 참고자료 쓰기 (Day 8)

1-1. 조각이 뭐였지 (세 번째 설명 — 이번엔 눈으로)

조각은 개념이 아니라 **파일 안의 문단**이야.

네가 쓸 파일:

```
rag_docs/위축형_칭찬법.md
──────────────────────────────────────
위축형 아이가 3번 이상 실패했던 동작을 해냈을 때는,
20개를 넘었다는 결과보다 "걸려도 계속 다시 한 것"을 콕 집어 칭찬한다.
결과를 칭찬하면 아이는 성공했을 때만 인정받는다고 느껴
다음에 못할 것 같으면 아예 시도하지 않게 되기 때문이다.
                              ← 빈 줄
위축형 아이가 강사의 심부름을 해냈을 때는, ...
                              ← 빈 줄
위축형 아이가 어린 동생을 챙겨줬을 때는, ...
──────────────────────────────────────
```

내일 프로그램이 이 파일을 읽으면 이렇게 돼:

```
chunks = [
    "위축형 아이가 3번 이상 실패했던 동작을 해냈을 때는 ...",   # 0번
    "위축형 아이가 강사의 심부름을 해냈을 때는 ...",            # 1번
    "위축형 아이가 어린 동생을 챙겨줬을 때는 ...",              # 2번
]
```

> **조각 하나 = 저 리스트의 원소 하나. 조각을 나눈다 = 파일에 빈 줄을 어디에 넣을지 정한다.**

그리고 나중에 검색할 때는 **저 중 딱 하나(또는 두 개)만** 뽑아서 프롬프트에 넣어. `0번`이 뽑히면 `1번`, `2번`은 **안 들어가.**

여기서 조각 규칙 두 개가 자동으로 나와:

왜냐하면	그래서 규칙
원소는 **통째로** 프롬프트에 들어간다	한 원소엔 **한 얘기만** (섞이면 답이 산만해짐)
원소는 **혼자** 들어간다 (앞뒤 문단 안 옴)	**"이럴 땐", "그 아이는"** 금지. 매번 주어를 다시 쓴다

1-2. 조각 하나 쓰는 골격

```
[언제 / 어떤 아이인지] → [뭘 한다] → [왜 그게 효과적인지]
```

세 번째 칸이 알맹이야. 왜냐면 AI가 학부모한테 이렇게 써야 하거든:

> "~해주세요. **왜냐하면 ~거든요.**"

세 번째 칸이 없으면 AI는 "칭찬해주세요"밖에 못 써. Day 4의 뻔한 피드백으로 돌아가.

1-3. 다 쓰고 스스로 검사하는 법

각 조각마다 두 개만 돌려봐:

- ① **요약 테스트** — "이 조각은 ___를 말한다" → 빈칸에 **하나만** 들어가면 통과
  - 두 개 넣고 싶으면 → **자른다**
  - 두 조각이 같은 빈칸이 나오면 → **합친다**
- ② **독립 테스트** — 이 조각만 떼어서 읽어도 무슨 말인지 아나?

문장 개수(2~4문장)는 **기준이 아니라 결과야.** ①②를 지키면 알아서 그만큼 나와.

1-4. 오늘 쓸 것

`rag_docs/위축형_칭찬법.md` **조각 1개.** 그거만.

재료는 이미 네가 골라놨어:

조각	언제	상태
0번	3번 이상 실패했던 **동작**을 해냈을 때	세 칸 다 찼음 → **오늘 이거만**
1번	강사가 시킨 **심부름**을 해냈을 때	내일
2번	어린 **동생**을 챙겨줬을 때	내일

---

2. 내일 오전 — 자료 4개 마저 (Day 8 마무리)

같은 골격으로 복사기처럼 쓰면 돼. 방향만 잡아둘게.

파일	조각에 꼭 들어갈 것	헷갈리지 말 것
승부욕형_훈육법.md	이기고 지는 것에 반응이 큰 아이 → 무엇을 기준으로 다시 잡아주나	"칭찬" 아니라 **훈육**
위축형_칭찬법.md	해냈을 때 어떻게 말하나	**사후** (이미 한 것)
위축형_적극성.md	아직 안 나설 때 어떻게 끌어내나 (예: **한 동작만 해보자**)	**사전** (아직 안 한 것)
평범형_성장법.md	눈에 띄지 않는 아이의 성장을 어떻게 보게 하나	

파일	조각에 꼭 들어갈 것	헷갈리지 말 것
학부모_전달_공통규칙.md	성향과 무관하게 **항상** 지킬 것	Day 4 [작성 규칙]에 박아둔 것들이 힌트

> 마지막 파일 팁: 지금 prompt_service.py의 [작성 규칙] 7개를 보면 "비교하지 마세요", "진단처럼 보이지 마세요" 같은 게 프롬프트에 **직접 박혀** 있어. 그중 자료로 빼는 게 나은 게 뭔지 내일 같이 보자. (지금 고민하지 마)

---

3. 내일 본편 — Day 9: 임베딩 · 저장 · 검색

여기가 RAG의 심장이야. **새 개념은 딱 두 개**야. 그것만 잡으면 돼.

3-1. 문제: 컴퓨터는 "비슷하다"를 모른다

수업기록에 이런 게 들어왔어:

> "*유리가 줄넘기를 3번 실패하고 결국 성공함*"

우리는 이게 조각 0번("3번 이상 실패했던 동작을 해냈을 때")이랑 **비슷하다**는 걸 한눈에 알아. 근데 컴퓨터한테 물어보면?

```
"유리가 줄넘기를 3번 실패하고 결국 성공함" == "위축형 아이가 3번 이상 실패했던 동작을..."
# False
```

글자가 다르니까 **다른 문장**이래. in으로 단어를 찾아볼 수도 있지만, "*매트 동작을 여러 번 못하다가 해냄*" 은 **"실패"라는 단어조차 없어.** 단어 찾기로는 못 잡아.

> **컴퓨터는 "같다/다르다"만 알지, "비슷하다"를 못 재.** 그래서 문장을 **숫자로 바꿔서, 거리를 잰다.** 그게 임베딩이야.

3-2. 새 개념 ①: 임베딩(embedding) = 문장을 숫자 좌표로 바꾸기

임베딩 = 문장을 넣으면 숫자 리스트가 나오는 함수.

```
embed("위축형 아이가 3번 실패한 동작을 해냈을 때 과정을 칭찬한다")
# → [0.021, -0.443, 0.187, ... ]   숫자 수백~수천 개짜리 리스트
```

이 숫자 리스트를 **벡터(vector)** 라고 불러. 어렵게 생각하지 마. **좌표**야.

- (3, 5) 는 2차원 좌표 → 종이 위의 점
- [0.021, -0.443, ...] 은 수백 차원 좌표 → **머릿속에 그리려 하지 마.** 그냥 "점"이라고만 생각해.

핵심 성질 딱 하나:

> **뜻이 비슷한 문장은 가까운 점이 된다.**

```
        ● "3번 실패 후 성공했을 때 과정을 칭찬"
      ●  "여러 번 못하다가 해냄"          ← 가까움! 단어는 하나도 안 겹치는데도


                  ● "줄넘기 손목 스냅은 손목만 돌린다"   ← 멀리 떨어짐
```

단어가 아니라 뜻으로 가까워져. 이게 임베딩이 마법처럼 보이는 이유야. (어떻게 그렇게 되는지는 몰라도 돼. 우리는 **가져다 쓰는 쪽**이야. Gemini가 만들어줘.)

3-3. 새 개념 ②: 유사도(similarity) = 두 점이 얼마나 가까운가

점 두 개가 있으면 **얼마나 가까운지 숫자로** 잴 수 있어.

```
similarity(질문_벡터, 조각0_벡터)  # → 0.87   가까움 ✅
similarity(질문_벡터, 조각1_벡터)  # → 0.31   멂
similarity(질문_벡터, 조각2_벡터)  # → 0.12   더 멂
```

보통 **-1 ~ 1** 사이 숫자가 나오고, **1에 가까울수록 비슷해.**

그럼 검색은 뭐냐? **그냥 이거야:**

> **조각 전부와 유사도를 재고, 제일 높은 놈을 고른다.**

끝이야. 벡터 검색이라는 게 대단한 게 아니라, **점수 매겨서 max 뽑기**야.

> 정식 명칭은 **코사인 유사도(cosine similarity)** 야. 두 점의 *방향*이 얼마나 같은지를 재는 방법인데, 내일은 이름만 알고 넘어가도 돼. 계산식은 내가 깔아줄게. (보일러플레이트)

3-4. 그래서 내일 만들 흐름

```
[준비 — 프로그램 켤 때 딱 한 번]
   rag_docs/*.md 읽기
        │
        ▼  빈 줄로 자르기
   chunks = ["조각0", "조각1", "조각2", ...]
        │
        ▼  embed() 를 조각마다 호출
   vectors = [[0.02, ...], [0.51, ...], ...]     ← 조각과 같은 순서!


[검색 — 피드백 만들 때마다]
   질문 = "유리가 줄넘기 3번 실패하고 성공함" (수업기록)
        │
        ▼  embed() 호출
   질문_벡터 = [0.03, ...]
        │
        ▼  vectors 전부와 유사도 계산 → 제일 높은 것 고르기
   → chunks[0]  ("위축형 아이가 3번 이상 실패했던 동작을...")
```

```
        │
        ▼
    build_prompt(child, records, rag_context=chunks[0])     ← Day 10
```

> `chunks`와 `vectors`가 **같은 순서**인 게 중요해. `vectors`에서 2번이 제일 가깝다고 나오면, 정답 글은 `chunks[2]`거든. 리스트 두 개가 번호로 짝지어져 있는 거야.

3-5. 내일 네가 칠 코드 (미리 채우지 말 것)

내가 깔아줄 것: 파일 읽기, 코사인 유사도 계산식, numpy 설치. **네가 칠 것: 함수를 부르고 결과를 꺼내는 줄.** 이 정도야.

```python
# rag_service.py  (내일 만들 파일)

def embed(text):
    """문장 하나 → 숫자 리스트"""
    response = client.models.embed_content(
        model=...,          # 내가 깔아줌
        contents=text,
    )
    return ...              # ← 네가 칠 곳: 응답에서 숫자 리스트 꺼내기


def search(question, chunks, vectors):
    """질문과 가장 비슷한 조각 1개를 돌려준다"""
    question_vector = ...   # ← 네가 칠 곳: embed() 를 부른다

    scores = []
    for v in vectors:
        scores.append(cosine_similarity(question_vector, v))   # 내가 깔아줌

    best_index = ...        # ← 네가 칠 곳: scores에서 제일 큰 값의 '번호'
    return chunks[...]      # ← 네가 칠 곳: 그 번호로 조각 꺼내기
```

정확히 4줄이야. 나머지는 다 보일러플레이트.

---

4. 내일을 위한 손풀기 — 함수 호출 (약한 부분)

`todayplan`에 "함수 호출 개념 다시 보기"가 남아 있었지. **내일이 바로 그날이야.** 위 코드에서 네가 칠 4줄이 전부 **함수 부르고 결과 받기**거든.

이미 해본 것 (Day 4~5)

```python
prompt = build_prompt(child, records)      # ① 부르고  ② 결과를 변수에 담음
result = generate_feedback(prompt)         # ① 부르고  ② 결과를 변수에 담음
```

이 두 줄에 함수 호출의 전부가 들어 있어.

```
prompt   =   build_prompt(  child, records  )
  ▲              ▲               ▲
  |              |               └── 인자(argument): 함수한테 주는 재료
  |              └──────────────── 함수 이름
  └─────────────────────────────── return 된 값이 여기 담긴다
```

딱 세 가지만 기억해:

1. `()` **를 붙여야 실행된다.** `build_prompt` 는 함수 그 자체고, `build_prompt(a, b)` 는 실행한 결과야.
2. **재료는 괄호 안에, 순서대로.** `build_prompt(records, child)` 로 쓰면 뒤바뀌어서 터져.
3. `return` **한 값은 받아야 남는다.** `build_prompt(child, records)` 만 쓰고 `prompt =` 를 안 붙이면 결과가 그냥 사라져.

30초 자가진단

내일 이 코드를 볼 거야. **지금 대답해봐:**

```
question_vector = embed(question)
```

- `embed` 에 주는 재료는 뭐야?
- 그 결과로 `question_vector` 에 담기는 건 뭐야? (숫자 하나? 리스트? 문장?)

막힘 없이 나오면 내일 4줄은 그냥 써져.

---

5. 내일 막히면 볼 것

공식 문서 (1순위)

- Gemini 임베딩: `ai.google.dev` 문서에서 **Embeddings** 항목
  - ⚠️ **모델 이름은 문서에서 직접 확인해.** 버전이 자주 바뀌어서 내가 여기 적어두면 틀릴 수 있어.
  - `client.models.embed_content(...)` 라는 **모양**만 기억하고, 정확한 인자는 문서 보고 채우자.

검색 키워드 (이렇게 그대로 쳐)

- `Gemini API embeddings python`
- `코사인 유사도 파이썬 numpy`
- `RAG chunking 전략` ← 우리가 손으로 한 걸 기계로 하는 법. 궁금하면 나중에.

유튜브

- `RAG 설명` 또는 `임베딩 벡터 검색 설명` 검색 → 10분짜리 개념 영상 1개면 충분

---

6. 내일 아침 자가진단 (이 3개 대답되면 바로 시작)

문서 덮고, **소리 내서** 대답해봐. 30초씩.

1. **RAG가 뭐야?** → (힌트: "넣는다"로 끝나면 반쪽이야. 앞에 뭐가 붙어야 하지?)
2. **조각(chunk)이 뭐야?** → (힌트: 개념 말고 **물리적으로.** 파일에서 뭘 보면 되지?)
3. **임베딩이 왜 필요해?** → (힌트: 컴퓨터가 못 하는 게 뭐라고 했지?)

세 개 다 되면 **자료 4개 쓰고 바로 Day 9.** 하나라도 막히면 그 섹션만 다시 읽어. 전체 다시 읽지 마. 그게 시간 낭비야.

부록 — 오늘까지 한 줄 요약

개념	한 줄
RAG	질문에 맞는 참고자료를 **찾아서** 프롬프트에 끼워주는 것 (찾는 게 절반)
chunk(조각)	파일에서 빈 줄로 나뉜 문단 하나 = 검색으로 뽑혀 나올 최소 단위
조각 기준	조각 1개 = 질문 1개에 대한 대답 (①요약 하나 ②혼자 읽혀야)
조각 골격	`[언제/어떤 아이]` → `[뭘 한다]` → `[왜 효과적인가]` ← 세 번째가 알맹이
임베딩	문장 → 숫자 좌표(벡터). 뜻이 비슷하면 가까운 점이 됨
유사도	두 점이 얼마나 가까운지 재는 숫자. 제일 높은 조각을 고르는 게 곧 검색
최종 목적지	`build_prompt` 안의 `{rag_context}` 칸을 채우는 것